When the Checker Is Noisier Than the Model: Measuring Number Faithfulness in LLM-Written Business Notes

Harshitha Venkatesh

Preprint, October 17, 2026

Abstract

Executives increasingly read weekly performance notes drafted by large language models (LLMs) from sales tables, where one wrong number can mislead a decision. We present NumGate, an open, reproducible harness that measures how many numbers in such a note the source table does not support (number level) and how many claims attach a real number to the wrong entity, metric or period (claim level), using a two-level rule-based gate whose verdicts carry the supporting cell or operation. NumGate builds 60 weekly report tables from the public UCI Online Retail II data and compares four generation strategies: direct writing, compute-then-write with code execution, write-then-verify, and a deterministic facts template. A pilot with Claude Opus 5 showed that the checker's own errors can dominate the measurement: the first version of the gate overstated the claim error rate about 16 times. We then ran a pre-registered full study with three Claude writers (Opus 5, Sonnet 5 and Haiku 4.5), 60 tables, four strategies and two repeats: 1,440 notes with 44,087 checked numbers. Every gate flag on the 54 tables the pilot did not use, and a random sample of unflagged numbers, was checked blind by two independent reviewers, which were AI agents with programmatic access to the tables rather than people. Confirmed errors are rare: 0.22% of checked numbers for Opus 5, 0.05% for Sonnet 5 and 0.47% for Haiku 4.5. Only 30.6% of the gate's flags were real errors, and its precision fell with the writer's error rate, to 7.6% for Sonnet 5, where one phrasing habit caused most false alarms. Raw gate rates would therefore rank Sonnet 5 below Opus 5, while confirmed errors reverse the order. Write-then-verify and the facts template reduced confirmed errors relative to direct writing for Opus 5 and Haiku 4.5. Compute-then-write increased Opus 5's errors, mostly through penny-level rounding and mislabelled directions introduced by the model's own facts script. None of 451 sampled unflagged numbers was wrong (95 percent upper bound 0.8%). At the error rates of current models, automatic faithfulness scores should be reported with the checker's precision measured on the same text.

1 Introduction

Weekly business reviews summarise tables: revenue by country this week (TW) versus last week (LW) and the same week last year (LY), week over week (WoW) and year over year (YoY) changes, shares and ranks. LLMs are now used to draft these notes. The failure that matters most in this setting is not fluency or tone but a wrong number: a revenue figure attributed to the wrong country, a decline reported as growth, or a total that no row of the table supports.

Measuring this failure is harder than it looks. A note can cite a number that is not a cell but is correctly derived (a share, a total, a percent change), cite a correct number with the wrong label (last year's figure presented as last week's), or state the right number for the wrong entity. Scoring notes by hand does not scale across models, strategies and table sizes, and an automatic checker

has its own error rate.

We ask four questions:

RQ1 What fraction of the numbers an LLM cites in an insight note are not supported by the table?

RQ2 What fraction of its claims bind a number to the wrong entity, metric or period?

RQ3 Which generation strategy reduces these rates most: direct writing, compute-then-write with code execution, or write-then-verify?

RQ4 How do the rates change with table size and model?

This paper describes the measurement instrument, a pilot study with one model on six tables that shaped the instrument, and a pre-registered full study with three models on all 60 tables that answers RQ1 to RQ4. Our contributions are:

- **NumGate**, a reproducible harness: a table builder over public retail data, four generation strategies behind one model interface with on-disk caching and a budget stop, a two-level number gate, analysis with bootstrap confidence intervals, and an interactive results dashboard.
- **A precise, rule-based number gate** whose rules fit in two pages (Section 5) and whose verdicts carry evidence (the table cell or the operation that supports each number).
- **Pilot results** for Claude Opus 5 across the four strategies, at the number, claim and note level, with every flag checked against the tables.
- **A pre-registered full study** (Section 9): 1,440 notes from Claude Opus 5, Sonnet 5 and Haiku 4.5, in which every gate flag and a random sample of unflagged numbers were reviewed blind, so that the reported error rates are confirmed errors rather than raw checker output.
- **A measurement lesson**, first seen in the pilot and confirmed at scale: a first version of the gate overstated claim errors by about 16 times, and the final gate's precision on real notes ranges from 43.1% to 7.6% depending on the writer, enough to reverse the ranking of two writers. We recommend that papers reporting LLM faithfulness rates also report the precision of their checker on the same text.

2 Related work

Faithfulness in data-to-text generation. Table-to-text generation has long been evaluated for faithfulness to the source. Wiseman et al. [13] introduced extractive evaluation of generated game summaries, including a relation-generation measure that extracts (entity, value, type) records from text and checks them against the source table, which is the closest precedent to our Level 2. ToTTo [12] and PARENT [4] address faithful table-to-text generation and its evaluation. TabFact [2] frames the verification of statements against tables as a classification task. Our setting differs in three ways: the text is an executive note with many derived quantities, the checker is fully deterministic apart from claim extraction, and we evaluate generation strategies rather than models alone.

Hallucination detection. Surveys document how often generated text states unsupported facts [8]. FActScore [11] decomposes long-form text into atomic facts and checks each, and SelfCheckGPT [10] detects hallucination from sampling consistency. NumGate follows the

decompose-and-check pattern, but its source of truth is a small table, so support can be decided exactly with a rounding rule.

Computing before writing, and verifying after. Program-aided language models [7] and Program of Thoughts [3] have the model write code whose execution produces the numbers, which motivates our compute-then-write strategy. Self-Refine [9] and Chain-of-Verification [5] have the model critique or verify its own draft; our write-then-verify strategy instead feeds back the flags of an external deterministic checker.

3 Task and data

Source. We use UCI Online Retail II [1]: invoice lines of a UK online gift retailer from December 2009 to December 2011, with invoice, stock code, description, quantity, date, unit price, customer and country. The data come as two yearly sheets that overlap in early December 2010; keeping each invoice once leaves 1,044,848 lines. We drop cancelled invoices, rows with non-positive quantity or price, rows without a country and non-product stock codes (postage, bank charges and similar), leaving 1,014,193 rows. Revenue is quantity times unit price, in GBP.

Report tables. We aggregate to ISO weeks. Each table lists the top N entities of one week by revenue, with columns Entity, Revenue_TW, Revenue_LW, Revenue_LY, Units_TW, Units_LW, Orders_TW, Orders_LW, WoW_Pct, YoY_Pct, Share_Pct and Rank. All derived columns are computed from the rounded stored columns, so every derived cell recomputes exactly from the CSV. Tables come in three sizes: small (5 to 10 rows), medium (20 to 40) and large (80 to 150). Countries fill only small tables (a typical week has about 13 trading countries), so medium and large tables list products. With a fixed seed we build 60 tables: 20 small (10 country, 10 product), 20 medium and 20 large.

4 Generation strategies

All strategies produce the same note format, enforced by the prompt: 6 to 8 bullet lines, one insight per line, each naming an entity and containing at least one number, with periods labelled explicitly (TW, LW, LY, WoW, YoY), facts only and no recommendations. All prompts are released with the harness.

S1 Direct. The table is given as a markdown table; the model writes the note in one call.

S2 Compute-then-write. The model writes a short pandas script that computes up to 25 facts (entity, metric, period, value, formula). The script runs in a subprocess with a 30 second timeout; on failure it is retried once with the error message. A second call writes the note from the facts alone, without the table.

S3 Write-then-verify. The S1 draft is scored by the Level 1 gate; unsupported numbers are sent back with the table for correction or removal, for at most two rounds.

S4 Facts-template. A deterministic pandas facts packet (totals, top three entities, largest WoW and YoY movers, most orders) is narrated with every number copied verbatim. S4 is an upper bound on what a gated design can reach, not a free-generation strategy.

5 The number gate

The gate is deterministic except for claim extraction. Full rules with worked examples are in the released method document; we summarise them here.

Level 1: number support. A single regular expression extracts every numeric mention with its value, cited precision, unit (currency, percent, count, plain), sign and hedge. Signs come from explicit signs (including an en dash used as a minus) or from direction words (fell, rose, down) before or after the number, but a level word (to, from, at) between a direction word and the number makes it a level: in "fell to £1,632.90 TW" the amount is unsigned. Numbers that are not measurements are skipped with a recorded reason: digits inside entity names, week labels, years and counting phrases ("top 3", "2 of the 5 countries"). A cited value c matches a true value t when t rounds half-up to it at the cited precision, $-h \le |t|/s - |c| < h$ with $h = 0.5 \cdot 10^{-d}$ for scale s and d decimals, and signed values agree in sign; values written with k, M or B also match within 0.5 percent, hedged values ("about") within 5 percent. Each number is compared, in order, with (1) cells of the entities named in its line, (2) values derived from those entities and from the groups and product families the line mentions (changes, shares, share-point changes, per-order values, totals of the rest of the table, family sums), (3) table-level aggregates when the line names no entity or uses an aggregate word, and (4) any other cell. The first match labels the number Supported_cell or Supported_derived with its evidence; otherwise it is Unsupported. Anchoring derived values on the entities named in the line keeps the candidate set small: in a 150-row table, unrestricted pairwise operations would match almost any number by chance.

Level 2: claim binding. A cheap model (Claude Haiku 4.5 in our pilot) extracts one claim per number, (entity, metric, period, value, direction, qualifier), with a fixed JSON schema and prompt. The extractor only transcribes; values are re-parsed by the Level 1 parser. Labels written in the note override the extractor: a period label adjacent to a value ("£2,729.22 LW") sets its period, change wording ("a decline of £51.1k") marks it as a change, and a unit noun ("8 orders") sets its metric; values are located only where they stand alone, so 33 is not found inside £680.33. Each claim is resolved to one or more rows (exact name, aliases and demonyms, word-subset match, fuzzy match with ties kept) and to a column, and labelled Correct; Wrong_value if only the sign differs or nothing matches; Wrong_metric if the value belongs to another column of the same row (for example the YoY figure cited as WoW); Wrong_entity if it belongs to the same column of another row that the line names, or of exactly one other row; and Unverifiable for names that fit several rows none of which matches and for metrics the table cannot answer. Group figures ("the three parasol lines combined", "the top five"), including subsets the extractor mislabels as the whole-table total, are checked against the same group, product-family, top-k and table totals that Level 1 uses for the line, and are labelled Correct or Wrong_value.

Metrics. The *unsupported rate* is Unsupported over checked numbers (RQ1). The *binding error rate* is (Wrong_entity + Wrong_metric) over verifiable claims (RQ2). The *claim error rate* adds Wrong_value. Rates are pooled over the notes of a group, with 95 percent percentile bootstrap intervals from 1,000 resamples of notes [6]; strategy differences use a paired bootstrap over conditions matched on table, model and repeat. The full study (Section 9) resamples tables instead of notes, because notes written on the same table are not independent.

6 Pilot study

Setup. The pilot runs Claude Opus 5 on the smoke set: two tables of each size (60-table pool, seeded selection), four strategies and two repeats, 48 notes in all. Opus 5 does not accept a temperature parameter and thinks adaptively by default, so repeats sample independently. Every response is cached, and the repeat index is part of the cache key. The pilot's note generation cost $2.54 ($0.053 per note on average).

Scale of the pilot. The 48 notes contain 1,730 numeric mentions: 1,625 were checked and 105 skipped (76 inside entity names, 21 counting phrases, 8 week labels). Of the checked numbers, 1,367 matched a cell and 255 a derived value. The extractor returned 1,613 claims, of which 1,613 were verifiable, including 69 group figures checked against group totals.

Table 1. Pilot results for Claude Opus 5 (48 notes; gate v3). Rates pool numbers or claims over notes; brackets give 95 percent bootstrap intervals over notes. With 12 notes per strategy the intervals are wide.

Strategy	Numbers	Unsupported %	Claims	Wrong entity or metric %	Any claim error %	Notes with an error	Cost per note
S1 Direct	514	0.6 [0.0, 2.0]	508	0.0 [0.0, 0.0]	0.6 [0.0, 2.2]	1 / 12	$0.049
S2 Compute-then-write	301	0.0 [0.0, 0.0]	301	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0 / 12	$0.095
S3 Write-then-verify	514	0.0 [0.0, 0.0]	508	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0 / 12	$0.053
S4 Facts-template	296	0.0 [0.0, 0.0]	296	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0 / 12	$0.015

Results. Table 1 gives the pilot rates. Opus 5 cited very few unsupported numbers: 3 of 514 under direct writing (0.6%), all in one sentence that reported a wrong total for a product family, and none under the other strategies. At the note level, which is what a reader of the note experiences, 1 of 12 direct-writing notes contained a wrong sentence and no note under the other strategies did. No claim under any strategy attached a number to the wrong entity, metric or period; the only claim errors are the same three group figures. Section 8 checks these verdicts against the tables. Write-then-verify revised 1 of 12 drafts: the draft that contained the pilot's only real error, a total for six parasol and umbrella lines. The revision did not correct that total; it replaced the claim with a narrower one, a total for the three EDWARDIAN PARASOL colourways only (£9,139.25 TW versus £9,454.25 LW, a 3.3% decline), which is correct. Removing or narrowing a flagged claim, rather than fixing it, is a behaviour a verify loop can reward. The facts template produced the shortest and cheapest notes. Table 2 breaks the pilot down by table size: all errors fall in one medium table, and with two tables per size no size effect can be read from it. 4 of 48 notes broke the 6 to 8 bullet rule, all on one small country table with six rows, where the model wrote fewer bullets than asked. These rates are far lower than the ones our first gate reported, which is the subject of the next section.

Table 2. Pilot results by table size (all four strategies pooled; two tables per size).

Table size	Tables	Notes	Numbers	Unsupported (%)	Claims	Claim errors	Notes with an error
small	2	16	440	0 (0.0)	438	0	0 / 16
medium	2	16	576	3 (0.5)	572	3	1 / 16
large	2	16	609	0 (0.0)	603	0	0 / 16

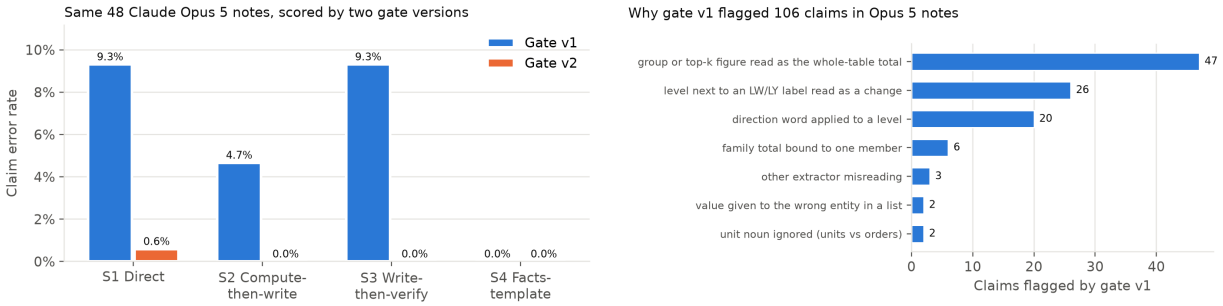


Figure 1. Left: claim error rate on the same 48 Opus 5 notes under gate v1 and gate v2. Right: why gate v1 flagged its 106 claims. Almost all v1 flags were false alarms caused by ordinary phrasing.

7 How much of the measured error is the checker's?

The first version of the gate (v1) was developed on a mock model that writes template notes with planted errors, where it caught every planted error. On the real Opus 5 notes it reported claim error rates of 9.3% (S1), 4.7% (S2) and 9.3% (S3), and an S1 unsupported rate of 2.1%. Reading the flagged sentences showed that nearly all were correct. Figure 1 categorises the 106 flagged claims:

- **Group or top-k figures read as the whole-table total** (47). "The top five TW products combined for 16.9% share" was extracted with entity TOTAL and compared with the total's 100 percent share.
- **A level next to an LW or LY label read as a change** (26). In "£3,205.71 TW versus £2,729.22 LW (WoW +17.5%)" the extractor gave the LW amount the period WoW.
- **A direction word applied to a level** (20). In "fell to £1,632.90 TW" the level was signed negative and failed to match.
- **Family totals bound to one member, unit nouns ignored, list order** (13 together): "the three EDWARDIAN PARASOL colourways combined" matched the BLACK product; "140 units and 8 orders" gave 8 the metric units; a "respectively" list was assigned out of order.

The fixes are small and general: directions sign only changes; a label written next to a value overrides the extractor; the extractor marks group figures as GROUP and Level 1 knows family and group totals; ambiguous short names are checked against every row they fit; and a same-row mix-up is tested before a different-row one. Gate v2 reports 0.6% for S1 on the same notes, and its Level 1 flags fell from 22 to 3.

The lesson generalises beyond this harness. When a model's true error rate is around one percent, a checker with a false alarm rate of a few percent per claim produces a measured error rate several times the true one, and differences between strategies can be artefacts of which phrasings each strategy tends to use. A checker validated only on synthetic errors (our mock) can look perfect and still fail on real text.

8 Validating the gate

The gate went through three versions. Gate v1 was built on the mock model. Gate v2 was revised after reading v1's flags on the pilot, and was then measured on a first set of fresh held-out notes. Gate v3 was revised using that held-out set's diagnoses and the pilot adjudication, frozen (its source file's checksum is recorded), and measured on a second held-out set written on tables no earlier test used. Each held-out set therefore measures a gate that was not revised on it; every other row of Table 3 measures a gate on text its rules were revised on. The full study then measures the frozen gate v3 on 1,440 real notes by blind review (Section 9).

Table 3. Accuracy of the gate on its test sets. False alarms are correct numbers (L1) or claims (L2) flagged as errors; caught errors are deliberately wrong (or, for the pilot, adjudicated wrong) numbers and claims that were flagged. Only the held-out rows measure a gate on text its rules were not revised on.

Test set	L1 false alarms (%)	L1 errors caught (%)	L2 false alarms (%)	L2 errors detected (%)
Regression suites, gate v3 (rules tuned on them)	2 / 291 (0.7)	61 / 88 (69)	0 / 149 (0.0)	49 / 49 (100)
Held-out set 1, gate v2	22 / 353 (6.2)	79 / 111 (71)	8 / 202 (4.0)	75 / 77 (97)
Held-out set 2, gate v3	7 / 435 (1.6)	90 / 146 (62)	6 / 270 (2.2)	99 / 104 (95)
Opus pilot, gate v3, adjudicated (rules revised on it)	0 / 1,622 (0.0)	3 / 3 (100)	0 / 1,610 (0.0)	3 / 3 (100)

Regression stress suites. Two suites written in varied LLM styles, with ground truth computed from the tables, test Level 1 on 379 numbers and Level 2 on 203 claims. Gate v3 raises 2 false alarms on 291 correct numbers and catches 61 of 88 wrong ones; most of the misses are real cells attached to the wrong row or period, which Level 1 cannot see by design and Level 2 catches. At Level 2 it raises 0 false alarms and detects 49 of 49 wrong claims. Because the fixes were developed while looking at these suites, we report them as regression tests, not as accuracy estimates.

Held-out set 1 (gate v2). Fresh notes written independently of the gate rules, on tables the suites never used: 21 notes on 5 tables (526 numeric mentions) for Level 1, and 8 notes with 294 extractor-style claims on 8 tables for Level 2. At Level 1 the gate raised 22 false alarms on 353 correct numbers (6.2%): 14 from gaps in the operation library (for example average order value for the whole table, or the rest of the table after the top k), 5 from rule gaps (for example "gave up" read as a rise) and 3 from entity names written with different spacing. It caught 79 of 111 wrong numbers (71%). Of the 32 misses, 21 are real cells attached to the wrong row or period, which Level 1 cannot see by design, and 11 are rule gaps: 6 direction verbs missing from the lists ("plummeted", "rebounded"), 4 hedged bounds and 1 number written as "c.137%". Excluding the by-design misses, it caught 88%. At Level 2 it raised 8 false alarms on 202 correct claims (4.0%), 4 of them from a single paraphrased product name matched to the wrong row, and detected 75 of 77 wrong claims (97%), with the error type right for 71 of 75. Of 10 group figures, 6 were verified and 4 were falsely flagged because their subsets are not in the library. The scripts for this set were written in a temporary folder that was later deleted, so only its recorded results survive; set 2 is

saved in the repository.

Held-out set 2 (gate v3). Fresh notes written by agents that did not see the gate's rules, following the note-format prompt, on 15 tables no earlier test used, frozen by checksum before gate v3 (also checksummed) scored them; files in `paper/heldout2/`. Level 1 (28 notes, 681 numeric mentions): 7 false alarms on 435 correct numbers (1.6%, Wilson 95% interval 0.8% to 3.3%), from a level after "down from" read as a change (3), plural or one-word family names (3) and the mean of the rest of the table, which the library lacks (2). It caught 90 of 146 wrong numbers: 87 of 98 value errors (89%; all 13 direction errors) but 3 of 48 binding errors, which Level 1 cannot see by design. Level 2 (18 notes, 417 claims written as a careful extractor would, so extractor errors are excluded): 6 false alarms on 270 correct claims (2.2%, all per-order averages read as revenue), 3 correct claims lost as Unverifiable (a missing alias, UAE), and 99 of 104 wrong claims detected (95%) with the error type right in all 99; 21 of 24 correct group figures and all 9 wrong ones were labelled correctly. Independent auditors re-derived a stratified random sample of 70 items per level from the tables and agreed with the recorded answer in all 140. Set 2 differs from set 1 in style (its notes follow the note-format rules, as real model output does) as well as in gate version, so the drop in false alarms from set 1 to set 2 is not a clean estimate of what v3's fixes achieved.

Adjudication of the pilot. We checked item by item against the tables, with AI assistance and programmatic lookups of each cell (not a human audit), every item the gate flagged in the pilot (3 unsupported numbers and 13 wrong claims under the gate at that point) and seeded random samples of 60 supported numbers and 60 correct claims. The 3 unsupported numbers were real errors, all in one sentence: a total for six parasol and umbrella lines reported as £14,211.15 TW versus £14,461.70 LW, a 1.7% decline, where the six rows sum to £17,211.15 and £16,325.70, a 5.4% rise. All 13 wrong claims were false alarms with three causes: group totals the extractor labelled TOTAL (6), a value found inside a longer number, such as 33 inside £680.33 (4), and absolute changes read as levels, such as "a decline of £51.1k versus LW" (3). The final gate fixes all three; on the pilot it flags exactly the 3 real errors at Level 1 and the same 3 as wrong group figures at Level 2, and nothing else. Because these fixes were made after seeing the pilot's flags, this agreement is not independent evidence of accuracy. The samples found no missed errors (0 of 60 numbers and 0 of 60 claims; a 95 percent upper bound of about 4.9 percent on the miss rate). One policy choice matters at this scale: six group shares equal the sum of the table's rounded Share_Pct cells but miss the exact share at the cited precision (for example 37.2% against an exact 37.13%); we accept them as derivable from the table as shown, and counting them as errors would raise the pilot's unsupported count from 3 to 9 numbers (from 0.18 to 0.55 percent of checked numbers).

Human audit. The harness also samples 200 numbers and 200 claims, stratified by strategy and label, for an audit by people and reports per-label precision and recall. No human audit has been done yet. The checks in this paper, including the full study's blind review, were made by AI agents with programmatic access to the tables, so a human spot check of a random subset of their verdicts is still needed before the error rates are treated as final.

9 Full study

Pre-registration. The analysis plan (`paper/preregistration.md`) was committed to the repository before any full-study note was generated. It froze gate v3 by checksum, together with the prompts, tables and code; fixed the design, the outcomes, four directional hypotheses and the statistics; and

named a primary set. The gate's rules were revised on the pilot's notes, and on the six pilot tables the full study's Opus 5 notes are the pilot notes themselves, returned from the cache, so the **primary set is the 54 tables the pilot did not use**. All 60 tables are a secondary analysis. Anything the plan does not list is labelled exploratory below.

Design. Three writers (Claude Opus 5, Claude Sonnet 5 and Claude Haiku 4.5, through the Anthropic API), the four strategies, 60 tables (20 per size) and two repeats give 1,440 notes. Opus 5 and Sonnet 5 do not accept a temperature, so they sample at the API default; Haiku 4.5 runs at temperature 0, so its two repeats are nearly identical. Claude Haiku 4.5 extracts the claims. No S2 script failed twice (0 failures) and no claim extraction failed (0), so nothing was excluded. The notes contain 46,866 numeric mentions, of which 44,087 were checked; the gate flagged 343. The API bill for generation and claim extraction was USD 39.86, under the USD 60 cap. Per-note costs add up to $58.56 because calls repeated from the pilot, and the S1 drafts that S3 starts from, were served from the cache but are counted in every note that uses them.

Blind review. The gate's flags were checked by review rather than trusted. A mention counts as flagged when Level 1 marks it Unsupported or Level 2 marks its claim Wrong_entity, Wrong_metric or Wrong_value. Review items were built for every flagged mention, for a seeded random sample of 300 mentions the gate supported and for 200 mentions whose claims it marked Correct (both stratified by writer), and shuffled. An item shows only the note's lines, the marked number and the path to the source table: not the writer, the strategy or the gate's verdict. Two reviewers worked through each batch of 60 items independently, one from the first item and one from the last, following a written guide (`paper/adjudication/REVIEW_GUIDE.md`) that fixes the rounding rule (a number must match at the precision it is written to), hedges, direction words and how derived values are recomputed. A third reviewer decided the items on which the first two disagreed. The reviewers were Claude agents, from the same model family as the writers, that read every table cell with pandas and wrote down their computation; they are not people. Of 843 items, the first two reviewers agreed on 99.5%. In the primary set, 4 flags were judged unclear; they are not counted as errors.

Outcomes. The main outcome is the *confirmed error rate*: flagged mentions the review judged wrong, over checked numbers. *Gate precision* is confirmed errors over flags, and the *miss rate* is the share of sampled unflagged mentions judged wrong. Intervals come from a cluster bootstrap that resamples tables (2,000 resamples, seed 42). Strategy effects are paired differences against S1 on the same tables, with a Holm correction across the three contrasts of each writer; writer effects are the same against Opus 5 within each strategy; the intervals are not adjusted for multiplicity, the p values are. Gate precision and the miss rate use exact (Clopper-Pearson) intervals.

Table 4. Full study, primary set (54 tables). Strategies: S1 Direct, S2 Compute-then-write, S3 Write-then-verify, S4 Facts-template. Gate flags: mentions the gate flagged, as a percentage of checked numbers. Confirmed: flags the blind review judged wrong. Precision: confirmed errors over flags. Notes with error: notes containing at least one confirmed error. Brackets give 95 percent intervals from resampling tables.

Writer	Strategy	Checked	Gate flags (%)	Confirmed	Confirmed (%)	Precision	Notes with error
Opus 5	S1	4,870	0.64 [0.33, 0.99]	6	0.12 [0.02, 0.24]	6/31	6/106
	S2	2,677	1.57 [0.90, 2.29]	27	1.01 [0.45, 1.66]	27/42	13/108
	S3	4,943	0.38 [0.16, 0.65]	1	0.02 [0.00, 0.06]	1/19	1/106
	S4	2,725	0.11 [0.00, 0.34]	0	0.00 [0.00, 0.00]	0/3	0/108
Sonnet 5	S1	3,888	1.00 [0.56, 1.54]	4	0.10 [0.03, 0.21]	4/39	4/108
	S2	2,329	0.56 [0.25, 0.91]	2	0.09 [0.00, 0.22]	2/13	2/108
	S3	3,887	0.90 [0.45, 1.44]	0	0.00 [0.00, 0.00]	0/35	0/108
	S4	2,685	0.19 [0.04, 0.37]	1	0.04 [0.00, 0.11]	1/5	1/108
Haiku 4.5	S1	3,869	1.40 [0.72, 2.16]	31	0.80 [0.30, 1.42]	31/54	18/108
	S2	1,770	2.03 [1.05, 3.11]	17	0.96 [0.38, 1.69]	17/36	15/108
	S3	3,845	0.73 [0.36, 1.13]	6	0.16 [0.00, 0.35]	6/28	6/108
	S4	2,412	0.50 [0.09, 1.01]	2	0.08 [0.00, 0.25]	2/12	2/108

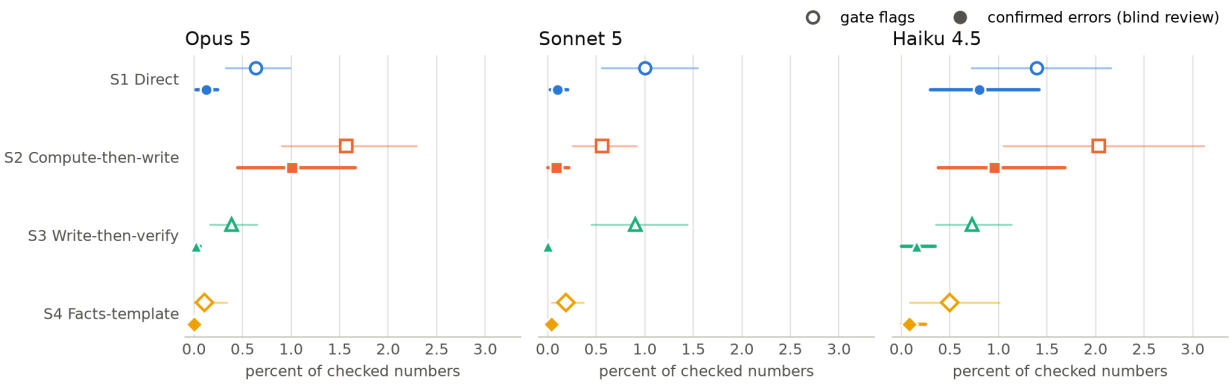


Figure 2. Gate flag rate (hollow markers) and confirmed error rate (filled markers) for each writer and strategy, primary set, with 95 percent intervals from resampling tables. The gap between the two markers is the gate's false alarms.

Error rates. Table 4 and Figure 2 give the results. Confirmed errors are rare for all three writers: 0.22% of checked numbers for Opus 5 (34 of 15,215), 0.05% for Sonnet 5 (7 of 12,789) and 0.47% for Haiku 4.5 (56 of 11,896). At the note level, which is what a reader experiences, errors concentrate in two cells: 18 of 108 Haiku 4.5 notes under direct writing and 15 of 108 under compute-then-write contained a wrong number, as did 13 of 108 Opus 5 notes under compute-then-write. Sonnet 5 wrote at most 4 of 108 notes with an error under any strategy. On all 60 tables (the secondary analysis) the rates are almost the same: 0.22%, 0.05% and 0.50%.

Table 5. Strategy contrasts in the confirmed error rate, primary set: each strategy minus direct writing (S1) on the same tables, in percentage points, with 95 percent intervals from resampling tables and *p* values corrected by Holm's method within each writer.

Writer	Contrast	Difference (points)	*p* (Holm)
Opus 5	S2 minus S1	0.90 [0.34, 1.59]	<0.001
	S3 minus S1	-0.10 [-0.21, -0.02]	0.018
	S4 minus S1	-0.12 [-0.24, -0.04]	0.018
Sonnet 5	S2 minus S1	-0.02 [-0.17, 0.15]	0.795
	S3 minus S1	-0.10 [-0.21, -0.03]	0.075
	S4 minus S1	-0.07 [-0.18, 0.06]	0.542
Haiku 4.5	S2 minus S1	0.16 [-0.73, 1.08]	0.725
	S3 minus S1	-0.65 [-1.20, -0.20]	0.003
	S4 minus S1	-0.72 [-1.35, -0.17]	0.012

Strategies. Table 5 compares each strategy with direct writing. For Opus 5 and Haiku 4.5, write-then-verify and the facts template both had fewer confirmed errors than direct writing: the differences are -0.10 and -0.12 points for Opus 5 (Holm $p = 0.018$ and 0.018) and -0.65 and -0.72 points for Haiku 4.5 ($p = 0.003$ and 0.012). For Sonnet 5, whose direct notes already had only 4 confirmed errors, no contrast survives the correction, although write-then-verify had none. Compute-then-write did not help any writer and raised Opus 5's error rate by +0.90 points ($p < 0.001$), making it Opus 5's worst strategy.

Hypotheses. **H1** (the facts template has the lowest confirmed error rate for every writer) is partly supported: it does for Opus 5 (0 errors in 2,725 numbers) and Haiku 4.5 (0.08%), but for Sonnet 5 write-then-verify had no error and the facts template one, a difference well within noise. **H2** (gate precision below 1, most of all for the strongest writer) is supported in its first part: precision was 35.8% for Opus 5 (95 percent interval 26.2% to 46.3%), 7.6% for Sonnet 5 (3.1% to 15.1%) and 43.1% for Haiku 4.5 (34.4% to 52.0%). The second part depends on a term the plan did not define. Precision was lowest for Sonnet 5, the writer with the fewest confirmed errors; if the strongest writer means Opus 5, its precision was not clearly lower than Haiku 4.5's. **H3** (under direct writing, more errors on large tables than on small ones) is not supported: large minus small was -0.08 points for Opus 5 (interval -0.40 to +0.26), +0.16 for Sonnet 5 (+0.00 to +0.40) and +0.57 for Haiku 4.5 (-0.60 to +2.10), with no *p* below 0.236. **H4** (under direct writing, Haiku 4.5 errs more than Opus 5) is supported: the difference is +0.69 points (+0.18 to +1.32, Holm $p = 0.012$).

Table 6. Kinds of confirmed error, primary set (exploratory; strategies as in Table 4). Rounding: a rounding or precision slip, off only in the last written digit, for example a share computed by adding already-rounded shares. Direction: the right magnitude called a decline when it was a rise, or the reverse. Value: a computed total, share or change that is off by more than rounding. Row or scope: a real value from the wrong row, column or period, or a claim stretched over rows it does not fit.

Writer	Strategy	Rounding	Direction	Value	Row or scope	Total
Opus 5	S1	6	0	0	0	6
	S2	20	7	0	0	27
	S3	1	0	0	0	1
	S4	0	0	0	0	0
Sonnet 5	S1	0	0	4	0	4
	S2	0	2	0	0	2
	S3	0	0	0	0	0
	S4	0	0	0	1	1
Haiku 4.5	S1	8	0	20	3	31
	S2	6	0	4	7	17
	S3	0	0	2	4	6
	S4	0	0	0	2	2

What the errors are (exploratory). After the review we sorted each confirmed error into one of four kinds, using the reviewers' written computations (Table 6; the labels are in `paper/adjudication/error_types_full.csv`). Of 97 confirmed errors in the primary set, 41 were rounding or precision slips, 9 had the wrong direction, 30 were wrong computed values (mostly Haiku 4.5's totals and top-ten shares under direct writing) and 17 used a real value from the wrong row, column or period, for example last year's revenue given as this week's. Opus 5's compute-then-write errors are 20 rounding slips and 7 wrong directions, and both come from the facts script the model wrote, not from the writing step. The script rounded money changes to one decimal (1213.7) and the writer printed them to the penny (£1,213.70 where the table gives £1,213.69); and it labelled the smallest of a set of rises the "largest drop", which the writer repeated. Computing before writing moves the risk into the code, where a careless rounding or label is copied faithfully into the note. Rounding slips are real errors at the precision written, but they are small. Setting them aside, the rates are 0.05% for Opus 5 (7 errors, all wrong directions under compute-then-write), 0.05% for Sonnet 5 and 0.35% for Haiku 4.5. The strategy contrasts keep their direction but none stays significant after correction (smallest Holm p 0.093), so the strategy effects above rest partly on rounding slips.

Table 7. The gate's own rates (pre-registered outcomes 1 and 2), primary set: unsupported numbers (Level 1), binding errors (Wrong_entity and Wrong_metric claims) and all claim errors (binding plus Wrong_value), as percentages, with 95 percent intervals from resampling tables; strategies as in Table 4. These are raw gate rates, before review.

Writer	Strategy	Numbers checked	Unsupported (%)	Claims verifiable	Binding errors (%)	Claim errors (%)
Opus 5	S1	4,870	0.45 [0.22, 0.73]	4,870	0.08 [0.02, 0.17]	0.51 [0.21, 0.87]
	S2	2,677	0.86 [0.33, 1.44]	2,676	0.37 [0.11, 0.68]	1.53 [0.89, 2.26]
	S3	4,943	0.20 [0.06, 0.40]	4,949	0.08 [0.02, 0.17]	0.24 [0.06, 0.45]
	S4	2,725	0.00 [0.00, 0.00]	2,724	0.11 [0.00, 0.34]	0.11 [0.00, 0.34]
Sonnet 5	S1	3,888	0.82 [0.49, 1.26]	3,891	0.18 [0.00, 0.52]	0.28 [0.05, 0.63]
	S2	2,329	0.17 [0.04, 0.35]	2,330	0.21 [0.00, 0.47]	0.39 [0.13, 0.73]
	S3	3,887	0.72 [0.37, 1.11]	3,891	0.18 [0.00, 0.51]	0.18 [0.00, 0.52]
	S4	2,685	0.07 [0.00, 0.19]	2,683	0.11 [0.00, 0.26]	0.11 [0.00, 0.26]
Haiku 4.5	S1	3,869	0.57 [0.21, 1.04]	3,937	0.66 [0.26, 1.13]	1.42 [0.71, 2.22]
	S2	1,770	0.90 [0.34, 1.64]	1,784	0.67 [0.00, 1.56]	1.91 [0.92, 3.01]
	S3	3,845	0.00 [0.00, 0.00]	3,907	0.67 [0.26, 1.16]	0.77 [0.35, 1.24]
	S4	2,412	0.25 [0.00, 0.51]	2,480	0.24 [0.00, 0.65]	0.24 [0.00, 0.64]

How good is the gate on real notes? Table 7 gives the gate's raw rates. Of its flags in the primary set, 97 of 317 were real errors (30.6%, 95 percent interval 25.6% to 36.0%). It missed little: none of the 451 sampled unflagged mentions was wrong (95 percent upper bound 0.8%), and one sampled mention was not a measurement. The per-writer samples are smaller, so the upper bounds on each writer's true error rate, adding the worst plausible misses to the confirmed errors, are loose: 2.61% for Opus 5, 2.44% for Sonnet 5 and 2.92% for Haiku 4.5.

Precision depends on how rare errors are and on how often the gate misfires on correct text. Using the false alarm rate measured on held-out set 2, the pilot predicted that only about 9% of flags would be real at its base rate (Section 10). On real notes the gate misfired less often than on our deliberately varied held-out notes: it flagged 0.40% of the numbers the review found correct in Opus 5's notes, so at a similar base rate Opus 5's precision was 35.8%. This supports the first of the two readings the pilot left open: real notes, constrained by the note format, use fewer of the phrasings that trip the gate. The misfire rate was higher for the other writers (0.66% for Sonnet 5 and 0.61% for Haiku 4.5), and with Sonnet 5's low error rate that left only 7.6% of its flags real. Its false alarms have a single main cause: of Sonnet 5's 62 Level 1 false alarms, 50 came from one phrasing, a level followed by a change, as in "TW revenue of £5735.62, down from LW £9020.33": gate v3 attaches "down" to the level before it, signs it negative and fails to match a correct cell. The same phrasing caused 6 of Haiku 4.5's 14 Level 1 false alarms and none of Opus 5's 26, and none of the flags it caused was a real error. Because of it, raw gate rates rank the writers in the wrong order. The gate flagged 0.72% of Sonnet 5's checked numbers and 0.62% of Opus 5's, but the confirmed rates are 0.05% and 0.22%. Within Sonnet 5, the gate flagged write-then-verify almost as often as direct writing (0.90% against 1.00%), although write-then-verify had no confirmed error. The gate stays frozen as pre-registered; the phrasing joins its known gaps (Section 10).

Reliability across repeats. Whether a note contains a confirmed error agreed between the two repeats of the same table and strategy with Cohen's $\kappa = 0.27$ for Opus 5 (93% agreement), -0.01 for Sonnet 5 (97%; errors are too rare for κ to say much) and 0.97 for Haiku 4.5 (100%). Haiku 4.5's high value reflects temperature 0, under which its two repeats are nearly the same note, not stability. For Opus 5, which samples, an error rarely recurred: of the 17 pairs of repeats in which a note had a confirmed error, both notes had one in only 3. Which note contains an error is largely chance, so single notes say little and rates should be pooled over many. Because the bootstrap resamples tables, the near-duplicate Haiku 4.5 repeats do not narrow its intervals.

10 Discussion and limitations

What the results say. Current Claude models write weekly notes whose numbers are almost always right: across strategies, fewer than one number in two hundred was wrong for every writer (at most 0.47%), and fewer than one in a thousand for Sonnet 5. Under direct writing, table size, from 5 to 150 rows, made no detectable difference. The errors that remain are not spread evenly. The smallest writer, Haiku 4.5, miscomputes totals and shares when it writes directly from the table, and uses real values from the wrong row or column. Feeding the gate's flags back (write-then-verify) or narrating a deterministic facts packet removed most of these. Asking the model to compute its facts in code first did not help, and for Opus 5 it introduced errors of its own: a script that rounds or labels carelessly passes its mistakes straight into the note. For a practitioner the lesson is to keep the arithmetic in code that is itself checked, as the facts template does, or to check the note against the table after writing, rather than to trust model-written code.

What the gate's own error rate implies. The pilot's true error rate (3 wrong numbers in 1,625, 0.18%) is far below the gate's false alarm rate on fresh held-out text: 6.2 percent of correct numbers and 4.0 percent of correct claims for gate v2 on set 1, and 1.6% and 2.2% for gate v3 on set 2. Bayes' rule makes the consequence concrete. With set 2's false alarm rate (1.6%) and value-error recall (89%), a Level 1 flag at the pilot's base rate would be a real error only about 9% of the time, roughly 10 false flags for every real one. The pilot's three flags were all real only because the gate's rules had been revised on those very notes. Two readings are consistent with our data: real model notes, constrained by the note format, may use fewer of the phrasings that trip the gate than our deliberately varied held-out notes do; or the pilot's zero false alarms reflect rules revised on the pilot itself. Either way, raw gate rates cannot be compared across models or strategies at this scale without an estimate of the gate's precision on the same notes. The full study bears this out. The first reading held for Opus 5, whose notes tripped the gate far less often than the held-out notes did, so the gate's precision on its notes (35.8%) was much higher than the pilot predicted. But one phrasing habit, which the held-out notes happened not to use, made most of Sonnet 5's flags false, and raw gate rates ranked the two writers in the wrong order. A checker's precision is therefore a property of the checker and the writer together; it has to be measured on the notes being scored, and it cannot be carried over from a validation set or from another model. We treat the gate as a triage tool whose flags are confirmed by review, and we report confirmed errors, not gate rates, as the main outcome.

Limitations.

- **The reviewers are not people.** Every check in this paper, including the full study's blind review, was made by Claude agents that recomputed each number from the table with code.

They agreed with each other on 99.5% of items, but they come from the same model family as the writers and could share blind spots, and their guide counts a number as wrong when it fails at the precision written, a strict rule a human reader might relax for a penny. A human spot check of a random subset of verdicts is still needed.

- **Errors the gate misses.** The review checked every flag but only a sample of 451 unflagged mentions. No sampled mention was wrong, but the per-writer samples are small, so the upper bounds on each writer's true error rate (2.61%, 2.44% and 2.92%) are far above the confirmed rates. Errors of a kind the gate cannot see, such as numbers written as words, are not counted at all.
- **One provider and three writers.** All writers and the claim extractor are Claude models, chosen for budget; other model families may err differently and trip the gate with other phrasings. Opus 5 and Sonnet 5 sample at the API default temperature while Haiku 4.5 runs at temperature 0, so their repeats are not comparable samples, and Haiku 4.5's error rate rests on what is effectively one note per table and strategy.
- **Exploratory analyses.** The kinds of error, the analysis without rounding slips and the phrasing behind Sonnet 5's false alarms were not in the pre-registered plan; the error kinds were assigned after the review, with the writer of each note known.
- **Pilot size.** 48 notes from one model on six tables; the intervals are wide and no strategy difference is significant. The final gate's rules were revised after seeing the pilot's flags, so its agreement with the adjudication on the pilot is not independent evidence; the held-out tests and the full study are.
- **One data domain.** A single UK online retailer, weekly grain, GBP, and country and product entities.
- **Known gaps in gate v3.** Held-out set 2 found: per-order averages cited as money are read as revenue at Level 2; a level after "down from" is read as a change; plural or one-word family names ("cushions", "MUG") are not recognised as families; the mean of the rest of the table is not in the library; bounds are not allowed on group figures; and some aliases (UAE) are missing. The full study added one more: a level followed by "down from" or a similar change phrase is signed negative. The gate stays frozen as pre-registered; fixing these gaps requires a new held-out set to measure the result honestly.
- **Gate coverage.** Numbers written as words ("doubled") are not extracted; derived values outside the operation library are Unsupported (the held-out tests found gaps such as table-level average order value and the rest of the table after the top k); some direction verbs ("plummeted", "rebounded") are missing from the lists; hedged bounds ("more than") can be satisfied by an unrelated column of the named row; paraphrased product names can bind to the wrong row; superlatives ("the largest swing") are not checked; the 0.5 percent tolerance for k and M values accepts a wrong last digit at large scales; truncated values without k or M count as unsupported.
- **Extractor dependence.** Level 2 inherits the extractor's errors, and a value assigned to the wrong entity in a list can still produce a false Wrong_entity.
- **S3 and its scorer.** Write-then-verify is revised against the Level 1 gate that also scores it, so its raw Level 1 rate is biased low (Haiku 4.5's is zero in Table 7, although the review confirmed 6 errors in its notes, all caught at Level 2). The confirmed error rate is the fairer comparison, but

it too only reviews what the gate flags.

11 Reproducibility

The harness is plain Python with pandas. Tables are rebuilt from the UCI download with a fixed seed and every derived column is checked on build. Every model response is cached on disk, keyed by model, prompts, temperature and repeat, so rerunning any experiment is free; a budget cap stops a run and a rerun resumes it. `run_all` builds the tables, runs a smoke experiment with a mock model (no API key), analyses it and opens the dashboard. The pilot is reproduced by `run_experiment.py --models claude-opus-5 --tables-per-size 2 --run-id smoke_opus`, the v1 comparison by `paper/rescore_v1.py`, and the pilot and validation numbers by `paper/make_numbers.py`. The full study is reproduced by `run_experiment.py --models claude-opus-5 claude-sonnet-5 claude-haiku-4-5 --run-id full`; its responses are in the released cache, so the rerun costs nothing. The pre-registered plan is `paper/preregistration.md`. The review items are built by `paper/adjudication/build_items.py`, reviewed by the workflow in `paper/adjudication/review_workflow.js` under `REVIEW_GUIDE.md`, and collected by `collect_verdicts.py`; the items, the unblinding key, both reviewers' verdicts with their computations, and the final verdicts are all released. `paper/full_study.py` computes every full-study number and table in this paper from the run and the verdicts.

References

[1] Daqing Chen. Online Retail II. *UCI Machine Learning Repository*, 2019. https://archive.ics.uci.edu/dataset/502/online+retail+ii

[2] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyou Zhou, and William Yang Wang. TabFact: A Large-scale Dataset for Table-based Fact Verification. *International Conference on Learning Representations*, 2020.

[3] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W. Cohen. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *Transactions on Machine Learning Research*, 2023.

[4] Bhuwan Dhingra, Manaal Faruqui, Ankur Parikh, Ming-Wei Chang, Dipanjan Das, and William Cohen. Handling Divergent Reference Texts when Evaluating Table-to-Text Generation. *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019.

[5] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-Verification Reduces Hallucination in Large Language Models. *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.

[6] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. *Chapman and Hall*, 1993.

[7] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided Language Models. *Proceedings of the 40th International Conference on Machine Learning*, 2023.

[8] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 2023.

[9] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-Refine: Iterative Refinement with Self-Feedback. *Advances in Neural Information Processing Systems*, 2023.

[10] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[11] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[12] Ankur P. Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. ToTTo: A Controlled Table-To-Text Generation Dataset. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 2020.

[13] Sam Wiseman, Stuart M. Shieber, and Alexander M. Rush. Challenges in Data-to-Document Generation. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 2017.